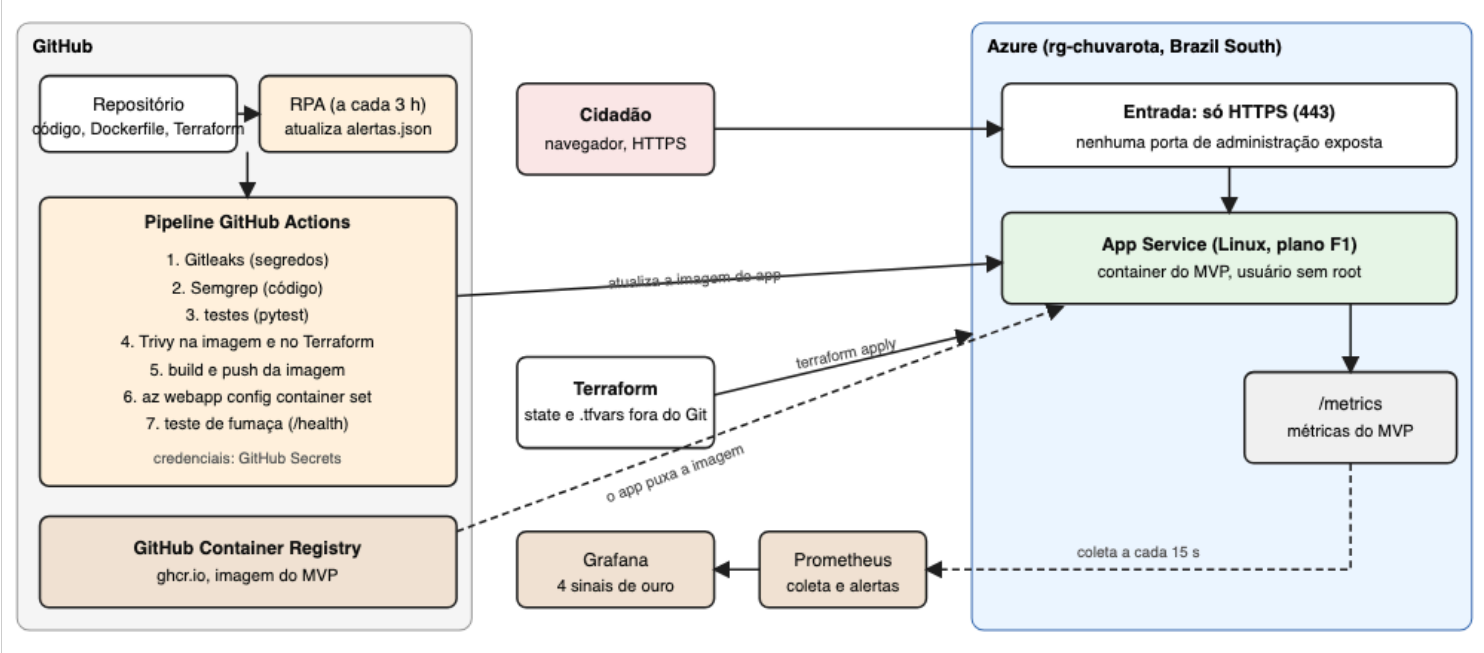
Secure DevOps & Cloud: deploy do ChuvaRota

Global Solution 2026.2 · Engenharia de Software · 4º ano

1. Integrantes e links

Vinicius Barros, RM 97824
MVP publicado: https://chuvarota-85ixn.azurewebsites.net
Repositório: https://github.com/ViniciusBarrosFiap/chuvarota (`Dockerfile`, `compose.yml`, `infra/`, `.github/workflows/devsecops.yml`, `monitoring/`)

2. Arquitetura



Fluxo do código à nuvem e do monitoramento.

Escolha da nuvem. Usei o Azure for Students. A assinatura "Starter" só permite alguns provedores de recursos (Web, SQL, Insights e poucos mais), então Container Apps, registro de imagens (ACR) e Log Analytics foram bloqueados. O MVP roda no **Azure App Service (Linux, plano F1 gratuito)**, com a imagem vinda do GitHub Container Registry (ghcr.io), que é público e gratuito.

3. Container e infraestrutura como código

3.1 Dockerfile e docker compose

- Imagem `python:3.12-slim`, 169 MB, com os pacotes do sistema atualizados.
- Roda como usuário `app` (uid 10001), sem root. Nenhum segredo entra na imagem (não há variável de ambiente sensível; o `.dockerignore` deixa de fora o que não é do MVP).
- `HEALTHCHECK` em `/health`. No `compose.yml` o MVP roda com sistema de arquivos somente leitura, `cap_drop: ALL` e `no-new-privileges`, junto com Prometheus e Grafana: `docker compose up --build`.

```
# docker
imagem chuvarota-mvp:latest  tamanho 169MB
uid=10001(app) gid=999(app) groups=999(app)
usuário da imagem: app  porta: map[8000/tcp:{}]
```

```
PYTHONDONTWRITEBYTECODE=1
PYTHONUNBUFFERED=1
```

3.2 Terraform

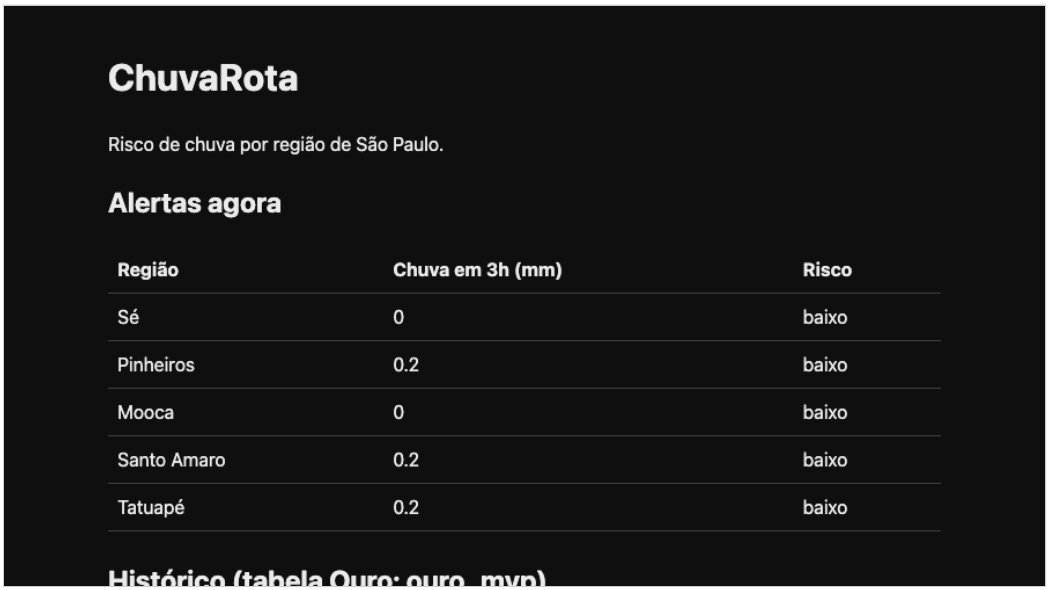
O Terraform (pasta `infra/`) criou o grupo de recursos, o plano e o app (`terraform apply`: 4 recursos). O state e o `terraform.tfvars` ficam fora do Git (`infra/.gitignore`); o repositório só tem o `terraform.tfvars.example`.

```
app = "chuvarota-85ixn"
grupo_de_recursos = "rg-chuvarota"
url_do_mvp = "https://chuvarota-85ixn.azurewebsites.net"
```

Porta de administração. Nenhuma porta de administração é exposta, para ninguém: o app só aceita HTTPS (o HTTP redireciona com 301), com TLS mínimo 1.2, FTPS desativado e publicação por FTP e Web Deploy com usuário e senha desligada no Terraform. Não existe regra de firewall com `0.0.0.0/0`.

```
# az webapp show (config de segurança)
{
  "apenasHttps": true,
  "app": "chuvarota-85ixn",
  "estado": "Running",
  "host": "chuvarota-85ixn.azurewebsites.net",
  "plano": "Free"
}
{
  "alwaysOn": false,
  "ftps": "Disabled",
  "imagem": "DOCKER|ghcr.io/viniciusbarrosfiap/chuvarota-mvp:7028a1c5c9a97b6421d3e9179f2fcaf5af8ac3b9",
  "tlsMinimo": "1.2"
}
Tipo                       Nome
-------------------------  ---------------
Microsoft.Web/serverFarms  plan-chuvarota
Microsoft.Web/sites        chuvarota-85ixn

{
  "camada": "Free",
  "plano": "plan-chuvarota",
  "sku": "F1"
}
```



MVP publicado no Azure.

4. Pipeline DevSecOps (GitHub Actions)

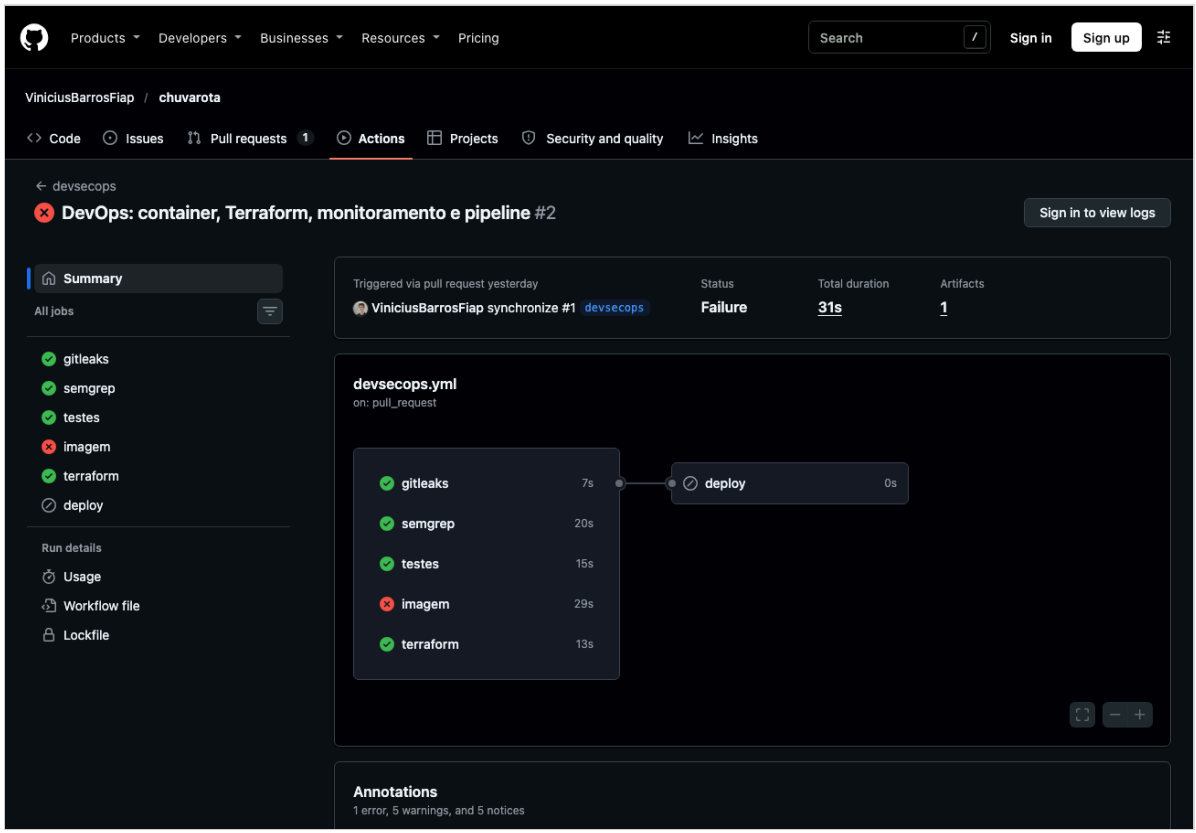
Etapa	Ferramenta	O que barra
gitleaks	Gitleaks	segredos no código e no histórico
semgrep	Semgrep (158 regras: Python e Dockerfile)	falhas no código
testes	pytest	regressões do RPA e do MVP
imagem	Trivy na imagem Docker	vulnerabilidades HIGH e CRITICAL com correção
terraform	Trivy (config) em `infra/`	configuração insegura na nuvem
publicar e deploy	docker, ghcr.io, Azure CLI	só roda se tudo acima passou; termina com teste de fumaça em `/health`

A credencial do Azure fica em **GitHub Secrets** (`AZURE_CREDENTIALS` , papel Website Contributor só no grupo de recursos) e o nome do app em Variables. O RPA, ao atualizar os alertas, dispara o pipeline para o MVP publicar os dados novos.

4.1 Execução barrada

Na primeira vez (prints abaixo), o Trivy reprovou a imagem por 2 falhas graves no pacote `apt` da imagem base, e o pipeline parou antes de publicar. Como o repositório foi recriado depois, refiz o teste no novo: coloquei de propósito uma versão antiga da biblioteca `urllib3` e o Trivy barrou com 8 falhas graves, sem publicar nada. https://github.com/ViniciusBarrosFiap/chuvarota/actions/runs/43599824816

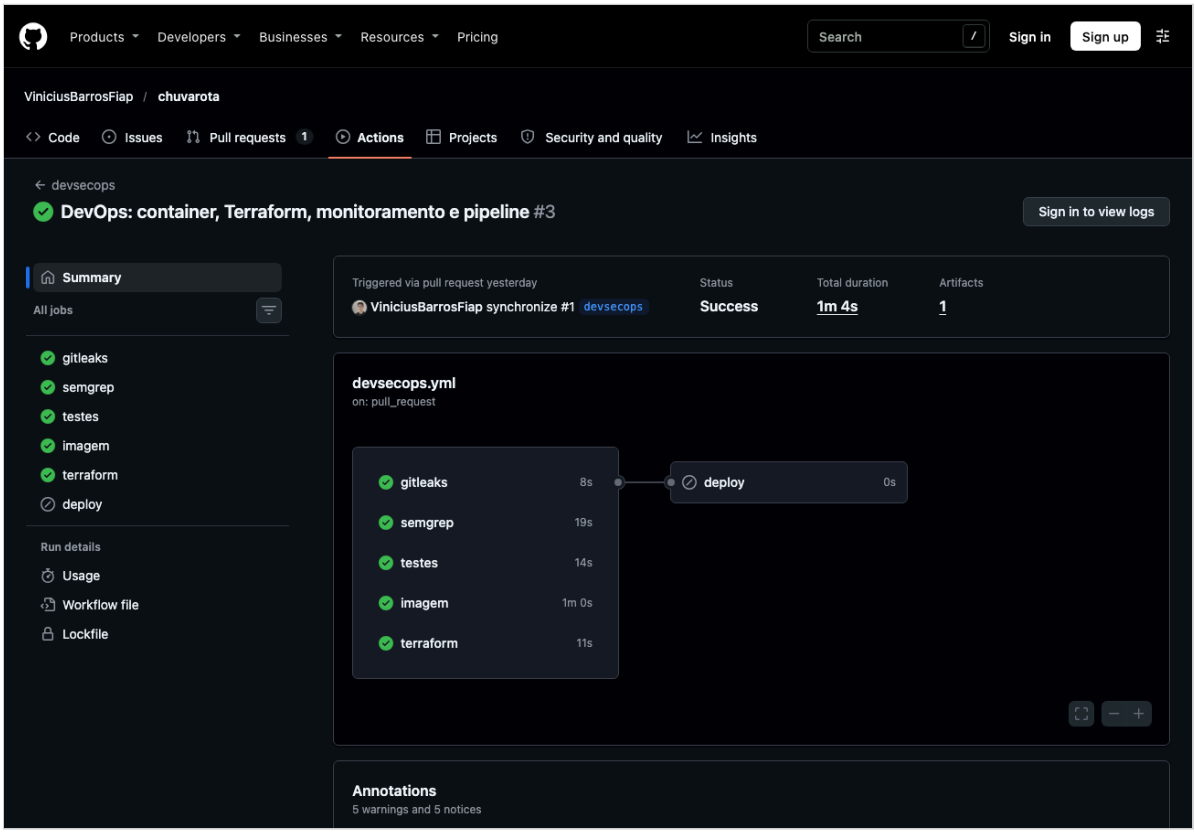
```
Total: 2 (HIGH: 2, CRITICAL: 0)
| apt           | CVE-2026-109518 | HIGH     | fixed  | 3.0.3            | 3.0.4         | On Debian, Ubuntu, and derivative L
inux distributions, an |
##[error]Process completed with exit code 1.
```



Execução reprovada: o job imagem falhou e o deploy foi pulado.

4.2 Execução corrigida

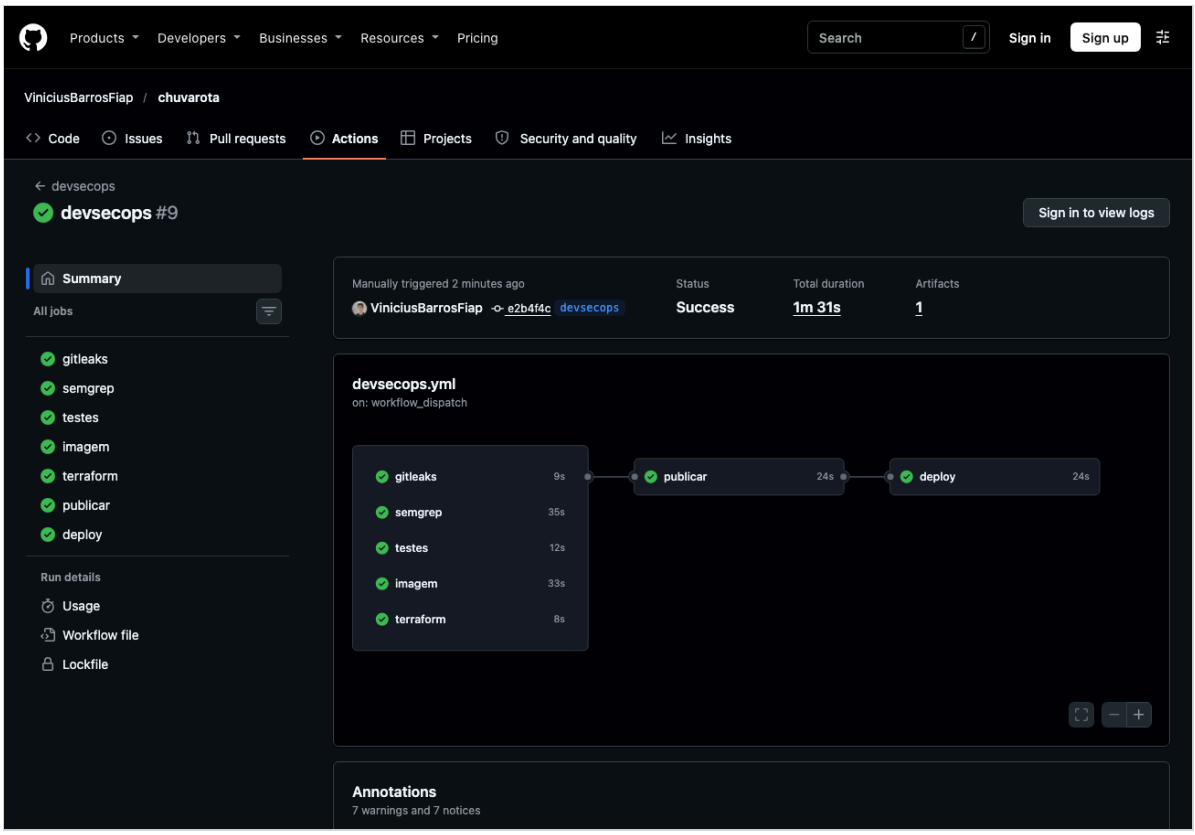
Na primeira vez, corrigi o Dockerfile (`apt-get upgrade`) e o pipeline passou em todas as etapas. No teste novo, tirei a biblioteca antiga e o pipeline passou de novo. https://github.com/ViniciusBarrosFiap/chuvarota/actions/runs/43599902680



Execução corrigida: todas as etapas de segurança aprovadas.

4.3 Deploy automático

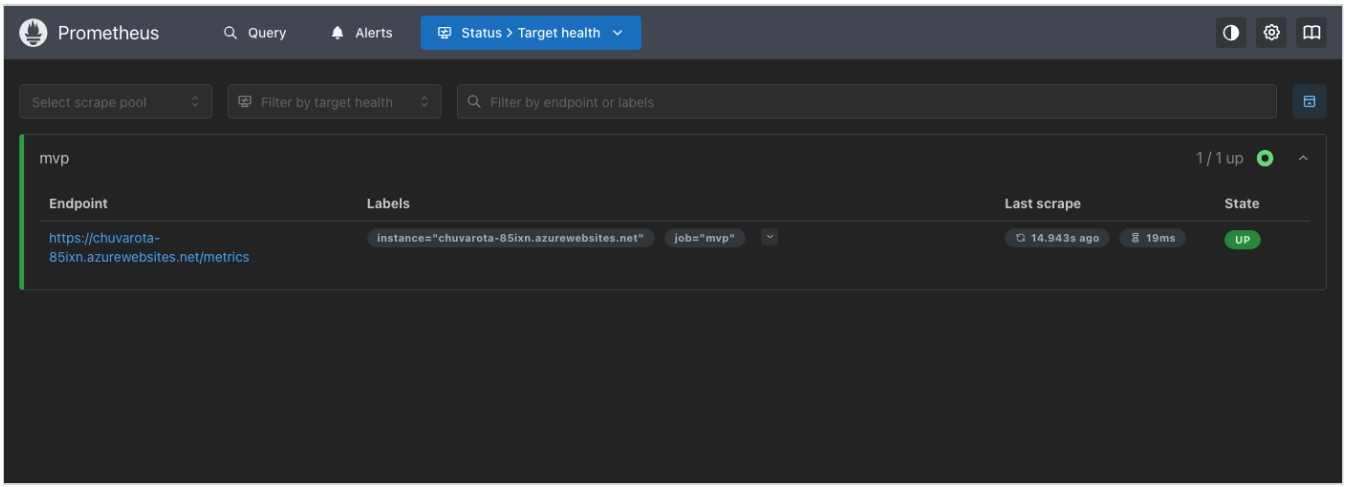
Com a credencial guardada em GitHub Secrets, o pipeline publicou a imagem no ghcr.io e atualizou o App Service sozinho, terminando com o teste de fumaça em `/health`. https://github.com/ViniciusBarrosFiap/chuvarota/actions/runs/43599724887



Execução com deploy: todas as etapas aprovadas, inclusive publicar e deploy.

5. Observabilidade

O MVP expõe `/metrics`. O Prometheus e o Grafana rodam localmente (`docker compose -f monitoring/compose.publicado.yml up`) e apontam para o **MVP publicado** no Azure, coletando a cada 15 segundos.



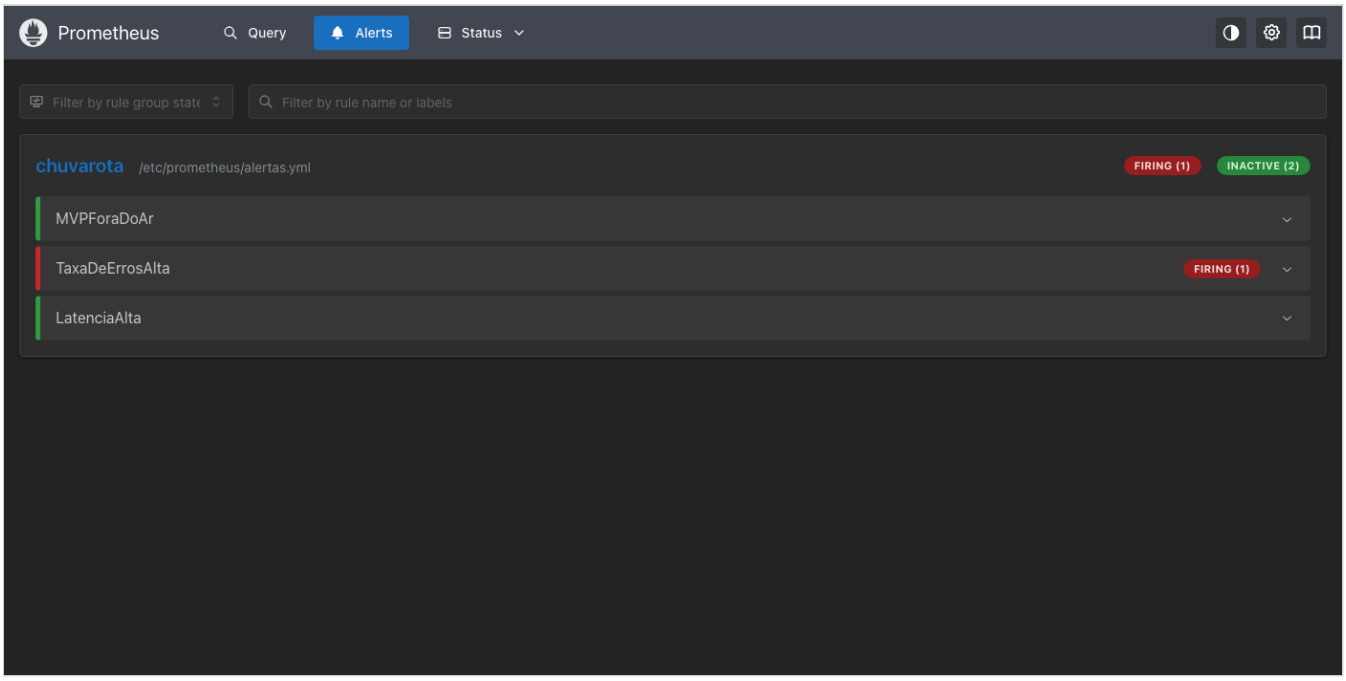
Prometheus coletando do MVP publicado (estado UP).

Sinal de ouro	Métrica
Latência	p95 de `http_request_duration_seconds`
Tráfego	pedidos por segundo por rota (`http_requests_total`)
Erros	proporção de respostas 4xx e 5xx
Saturação	memória do processo e pedidos em andamento



Painel do Grafana com os 4 sinais de ouro, durante um teste com tráfego e erros.

Alerta disparado. A regra `TaxaDeErrosAlta` (mais de 10% de erros por 30 segundos) disparou quando mandei pedidos válidos e inválidos ao MVP publicado. Há também `MVPForaDoAr` e `LatenciaAlta`.



Alerta TaxaDeErrosAlta em FIRING.

6. Responsabilidade compartilhada e LGPD

Camada	Quem cuida

Datacenter, rede física, hipervisor, sistema operacional e atualização do App Service, terminação do HTTPS	Microsoft (Azure)
Imagem e dependências, código do MVP, configuração do app (HTTPS, TLS, FTP), pipeline, segredos e acessos ao GitHub e ao Azure	Equipe
Dados tratados no MVP e o cumprimento da LGPD	Equipe (controladora)

Onde ficam dados pessoais. O MVP não pede cadastro e não guarda dados pessoais: mostra previsão de chuva por região e histórico agregado, e as leituras do sensor ficam só em memória, sem identificação de pessoa. O único dado possivelmente pessoal é o endereço IP de quem acessa, que passa pela plataforma do Azure (Brazil South) e não é gravado pelo MVP. Não configurei o Log Analytics (bloqueado na assinatura); se os logs HTTP do App Service forem ligados, o IP passa a ser registrado e a retenção deve ser definida.

7. Estimativa de custo

Item	**Custo**
App Service plano F1 (Brazil South)	US$ 0 (gratuito)
GitHub Container Registry e GitHub Actions (repositório público)	US$ 0
Prometheus e Grafana (rodando na máquina da equipe)	US$ 0
Total do que está em uso	**US$ 0 por mês**

Limites do plano gratuito: cota de CPU por dia (cerca de 60 minutos) e sem "always on" (o app dorme depois de um tempo sem uso, e o Prometheus coletando a cada 15 s ajuda a mantê-lo acordado). Se for preciso mais estabilidade, trocar para o plano B1 custa cerca de US$ 13 por mês (valor de tabela aproximado; confirmar na calculadora do Azure) e cabe no crédito de US$ 100 do Azure for Students: `terraform apply -var sku=B1`.

8. Reprodução

`az login`, copiar `infra/terraform.tfvars.example` para `terraform.tfvars` com o id da assinatura, `terraform apply` em `infra/` e `infra/configurar_github.sh`. Detalhes em `infra/README.md`.